UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Carrera de Computación / Software

Asignatura: Sistemas Distribuidos (Semana 3)

TA-IND-01 — Informe Académico Individual

Análisis Crítico de MinIO como Sistema de Almacenamiento de Objetos Distribuido y Comparación con Apache HDFS

Estudiante: **Heiner Ariel Zambrano Ronquillo**

Paralelo: **Séptimo Semestre "A"**

Fecha de Entrega: 13 de noviembre de 2026

Repositorio público institucional de GitHub:

https://github.com/Luispaca2002/TA-IND-01-Informe-Acad-mico-Individual-An-lisis-Cr-tico-de-Sistemas-Distribuidos-Unidad-1-

Directorio del estudiante: `TA-IND-01-Heiner Ariel Zambrano Ronquillo`

Resumen

El almacenamiento distribuido de datos requiere resolver simultáneamente la consistencia de metadatos, la disponibilidad ante caídas de hardware y la eficiencia en capacidad. En este informe analizo la arquitectura interna de MinIO en modo distribuido simétrico y la contrasto críticamente frente a Apache HDFS. El marco de análisis integra los fundamentos de la Unidad 1: transparencias de distribución, Teorema CAP/PACELC, modelos de comunicación y tolerancia a fallos. La investigación se respalda empíricamente en un clúster local de cuatro nodos contenerizados en Docker con imágenes endurecidas de Chainguard, configurando un conjunto de borrado (*Erasure Set*) mediante algoritmo Reed-Solomon con factor de redundancia $K = 2, M = 2$ (`EC:2`). Mediante el cliente oficial `mc`, se verificó la ingesta simétrica de objetos y se provocó la detención deliberada del nodo coordinador inicial (`docker stop minio1`). La prueba demostró que el clúster retuvo quórum estricto ($3/4 \geq 2$), preservando la lectura ininterrumpida de objetos mediante reconstrucción matemática por paridad, mientras que en el cliente se evidenció la ausencia de transparencia de fallos por dependencia de DNS estático. Tras reiniciar el nodo, el sistema ejecutó su autorrecuperación (*self-healing*) en 21 segundos. La comparación con HDFS fundamenta las ventajas de la arquitectura *Shared-Nothing* frente a la centralización del NameNode.

Palabras clave: MinIO, Erasure Coding, Reed-Solomon, Apache HDFS, Quórum, Tolerancia a Fallos, Consistencia Fuerte, Shared-Nothing.

1. Introducción

La evolución de las arquitecturas de software modernas hacia microservicios desacoplados, inteligencia artificial y canalizaciones analíticas masivas ha modificado las demandas impuestas a los sistemas de persistencia distribuida. Tradicionalmente, infraestructuras del ecosistema Big Data se construyeron sobre sistemas de archivos distribuidos jerárquicos como Apache HDFS (Hadoop Distributed File System). Sin embargo, la dependencia de un nodo maestro centralizado para la administración del árbol de directorios y la sobrecarga espacial de la replicación triplicada ($3\times$) representan limitaciones severas de escalabilidad y costo en la nube contemporánea [1], [2].

Para afrontar esta problemática surge MinIO, un sistema de almacenamiento de objetos de código abierto de alto rendimiento, completamente compatible con la interfaz de programación de aplicaciones Amazon S3 API. MinIO está diseñado bajo una premisa simétrica y sin estado maestro (*Shared-Nothing*), orientándose al almacenamiento de objetos no estructurados como registros analíticos, imágenes, modelos de aprendizaje automático y copias de seguridad a gran escala [3]. Empresas globales y organizaciones de investigación de vanguardia como Uber, Nutanix, Splunk y el CERN operan clústeres MinIO a escalas de cientos de petabytes, alcanzando tasas de transferencia que saturan enlaces de red de alta velocidad y buses de almacenamiento NVMe-oF [4], [5].

La selección de MinIO como objeto principal de este estudio responde a su relevancia como estándar de facto en almacenamiento de objetos para nubes privadas y su contraste directo con Apache HDFS. Mientras HDFS divide archivos en bloques secuenciales coordinados por

un NameNode central, MinIO dispersa objetos completos mediante codificación de borrado (*Erasure Coding*) directamente entre nodos pares.

Para validar estos principios, en este informe se documenta un laboratorio experimental reproducible compuesto por un clúster de cuatro nodos MinIO en Docker. Los resultados experimentales se cruzan sistemáticamente con literatura académica indexada reciente (2021–2026), articulando una discusión rigurosa en torno a las garantías de consistencia, los cuellos de botella de red y los compromisos de diseño (*trade-offs*). Debe aclararse que el laboratorio local sobre un único anfitrión físico permite evaluar la lógica del quórum y la conmutación de procesos, pero no reproduce el comportamiento de enlaces transcontinentales ni averías mecánicas independientes.

2. Arquitectura del Sistema MinIO

2.1. Componentes Clave y Modelo Simétrico

A diferencia de los sistemas de archivos jerárquicos convencionales, MinIO prescinde por completo de metadatos centralizados. Cada instancia del clúster ejecuta exactamente el mismo binario compilado en Go, asumiendo un diseño estrictamente simétrico (*Shared-Nothing*). La arquitectura se articula en torno a los siguientes componentes:

1. **Clientes S3 y Herramienta Administrativa (`mc`):** Cualquier cliente estándar compatible con AWS S3 SDK (en Python, Java, Go o cURL) o la interfaz de línea de comandos `mc` interactúa con el clúster a través de llamadas HTTP/HTTPS REST hacia los puertos S3 (predeterminado 9000).

2. **Capa de Balanceo de Carga (L4/L7):** En despliegues de producción se sitúa un balanceador como HAProxy o NGINX frente al clúster, proporcionando una dirección IP virtual única (*VIP*) y distribuyendo las conexiones entre todos los nodos disponibles mediante algoritmos *round-robin* o *least connections*.

3. **Nodos de Almacenamiento y Coordinador Efímero:** Cada nodo del clúster puede recibir indistintamente cualquier petición de lectura o escritura. El nodo que recibe la conexión HTTP asume temporalmente el rol de *coordinador efímero* para esa transacción específica, dispersando la carga y eliminando puntos únicos de congestión.

4. **Conjuntos de Borrado (*Erasure Sets*) y Unidades de Disco:** El clúster organiza el almacenamiento en conjuntos deterministas de discos denominados *Erasure Sets*. En nuestro laboratorio, los cuatro nodos conforman un conjunto de borrado con tamaño de banda 4 (*stripe size* = 4), aplicando un esquema Reed-Solomon con $K = 2$ bloques de datos y $M = 2$ bloques de paridad.

5. **Metadatos Atómicos (`xl.meta`):** MinIO almacena los metadatos del objeto (suma de comprobación SHA-256, marca de tiempo, etiquetas y mapa de partes) dentro del archivo atómico `xl.meta`, ubicado en el mismo directorio de almacenamiento de los fragmentos de datos, eliminando la necesidad de una base de datos externa como MySQL o etcd [4].

2.2. Flujo de Comunicación y Diagrama Formal

El procesamiento de una solicitud de escritura ilustra la coordinación interna del sistema:

1. El cliente envía una petición `PUT /bucket/objeto` al nodo coordinador (por ejemplo, `minio1`).
2. El coordinador calcula en flujo continuo (*streaming*) los fragmentos de datos y paridad aplicando la matriz de Vandermonde / Cauchy de Reed-Solomon.
3. Concurrente y sincrónicamente, el coordinador escribe el fragmento D_1 en su unidad local y despacha los fragmentos D_2, P_1 y P_2, junto con `xl.meta`, hacia los nodos pares (`minio2`, `minio3`, `minio4`) a través de llamadas a procedimientos remotos internas (RPC) sobre la red distribuida.
4. Una vez que la mayoría estricta de nodos confirma la escritura exitosa (quórum $\geq K+1=3$), el coordinador retorna el código HTTP 200 OK al cliente.

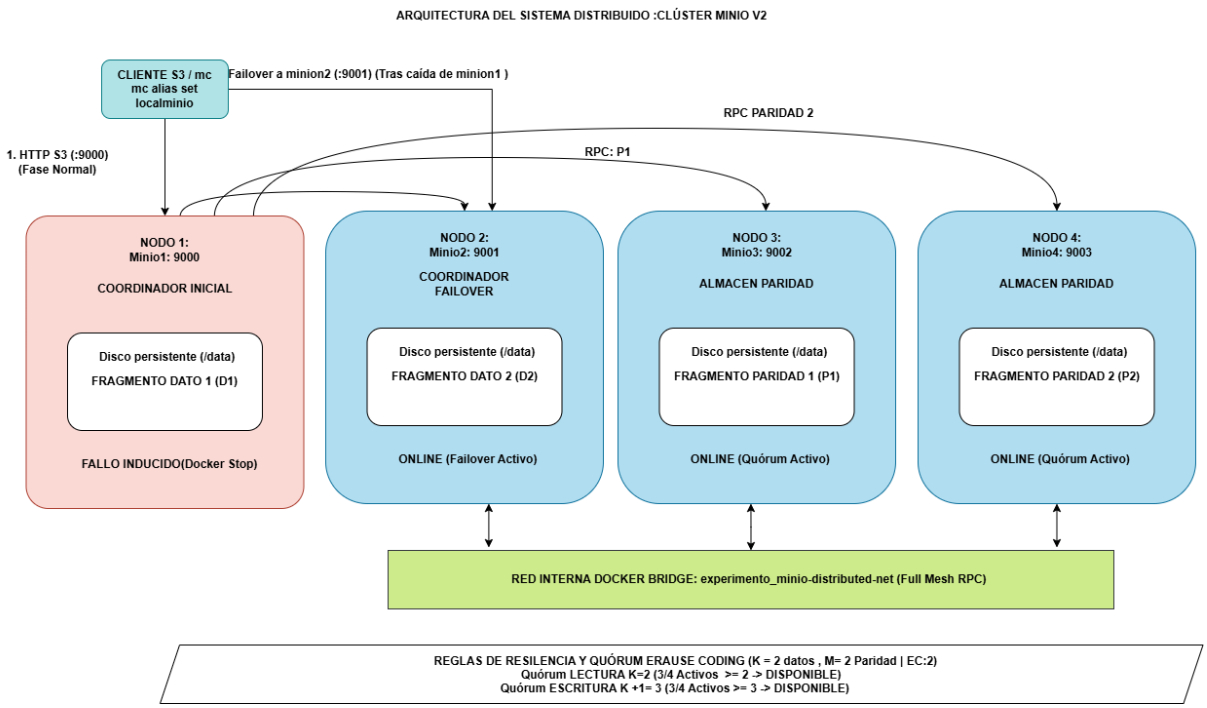


Figura 1: Diagrama de la arquitectura formal del clúster MinIO distribuido elaborado por el estudiante en Draw.io (`diagrama_MINIO.drawio.xml`), ilustrando la topología contenerizada de cuatro nodos, el flujo de dispersión de bloques de datos y paridad ($EC:2$) mediante RPC y el mecanismo de conmutación de cliente ante la caída del coordinador.

El diagrama de la Figura 1 representa la arquitectura formal del laboratorio y su evolución técnica respecto al bosquejo preliminar dibujado a mano alzada (conservado como Anexo A). En dicho esquema se resalta el rol del coordinador inicial, la ruta de conmutación por error (*failover*) hacia `minio2` ante la caída de `minio1`, y la malla de interconexión RPC en la red bridge privada `minio-distributed-net`.

3. Análisis con los Conceptos de la Unidad 1

3.1. Transparencias de Distribución

El concepto de transparencia busca ocultar a las aplicaciones usuarias la complejidad y distribución espacial de los recursos subyacentes. En la Tabla 1 se sintetiza la evaluación de las transparencias analizadas en MinIO, contrastando las funciones documentadas con lo comprobado en nuestro entorno.

Tabla 1: Evaluación de las transparencias de distribución en el clúster MinIO estudiado.

Transparencia	**Grado**	**Evidencia Experimental y Sustento Documental**
Acceso	Total	Los clientes leen y escriben objetos mediante llamadas HTTP REST estándar compatibles con S3, abstrayendo el cálculo matemático subyacente de Reed-Solomon [3].
Ubicación	Total	El cliente desconoce en qué nodos físicos o rutas locales de disco se almacenan los fragmentos D_1, D_2, P_1, P_2; solo referencia el nombre del bucket y la clave del objeto.
Migración	Parcial	Los objetos pueden reubicarse entre distintos *server pools* mediante políticas de ciclo de vida (*tiering*), pero no de forma dinámica a nivel de proceso en ejecución.
Reubicación	No soportada	MinIO no traslada fragmentos activos de un disco a otro mientras existe una transferencia en curso sin que el cliente deba reintentar la operación.
Replicación	Total	El usuario percibe una única entidad de datos atómica, mientras que el clúster dispersa físicamente múltiples fragmentos de datos y paridad en los cuatro discos.
Concurrencia	Total	Se admiten múltiples lecturas y escrituras simultáneas sobre distintos objetos mediante bloqueo determinista de metadatos por hash (*namespace locking*) [6].
Fallos	Parcial	**Comprobación crítica:** La reconstrucción interna del dato con un nodo caído es totalmente transparente; no obstante, el acceso externo falla si el cliente depende de un endpoint estático sin balanceador L4/L7.
Rendimiento	Parcial	MinIO escala linealmente al incorporar nuevos *server pools*, pero la saturación del bus de red o de la CPU por cálculo de paridad impacta en la latencia global [2].

El hallazgo central radica en la transparencia de fallos: mientras MinIO asegura la disponibilidad del contenido en modo degradado a nivel de almacenamiento, la transparencia de acceso para las aplicaciones cliente no es intrínseca al binario, sino que depende de una infraestructura de red perimetral que proporcione conmutación por error transparente.

3.2. Teorema CAP y PACELC

El Teorema CAP delimita los compromisos fundamentales de un sistema distribuido en presencia de particiones de red. Bajo esta formulación, MinIO se clasifica formalmente como un sistema **CP** (Consistencia y Tolerancia a Particiones), priorizando la corrección de los datos sobre la disponibilidad incondicional:

- **Consistencia en Partición (*C* sobre *A*):** Si una partición de red fragmenta el clúster aislando

nodos, MinIO exige que cualquier operación de escritura reúna un quórum estricto de más de la mitad de las unidades del conjunto de borrado ($Q_w \geq (N/2)+1$). En nuestro clúster de $N = 4$ unidades, la mayoría requerida es 3. La partición minoritaria (1 o 2 nodos) rechaza inmediatamente las solicitudes de escritura con error HTTP 503 (*Slow Down / Insufficient Drives*), evitando divergencias en el estado y previniendo el síndrome de cerebro dividido (*split-brain*) [1].

- **Ampliación PACELC (PC/EC):** La formulación PACELC estipula: si hay Partición (*P*), se elige Consistencia (*C*) sobre Disponibilidad (*A*); de lo contrario (*Else*, *E*), se prioriza Consistencia (*C*) sobre Latencia (*L*). MinIO implementa consistencia inmediata *read-after-write*: tras una escritura exitosa confirmada por el quórum, cualquier lectura subsiguiente desde cualquier nodo garantiza retornar la versión más reciente. Para ello, el coordinador debe recopilar y validar los metadatos `xl.meta` de al menos $K = 2$ nodos antes de despachar el objeto, aceptando el coste de latencia que impone la verificación por RPC en beneficio de la integridad estricta [3], [6].

3.3. Modelos de Comunicación

La arquitectura de MinIO combina múltiples modelos de comunicación distribuidos:

1. **Paradigma Estructural Híbrido:** Externamente opera como un modelo **Cliente-Servidor** sin estado, donde los clientes consumen endpoints REST sobre HTTP/1.1 y HTTP/2 mediante autenticación HMAC-SHA256 (*AWS Signature Version 4*). Internamente, los nodos se organizan en una malla **Peer-to-Peer simétrica**, donde cada nodo mantiene conexiones persistentes con los demás y actúa como par equivalente.

2. **Sincronismo de Operaciones:**

   - *Comunicación Sincrónica:* Las operaciones de lectura y escritura de objetos son bloqueantes y sincrónicas; el cliente aguarda la respuesta HTTP hasta que el quórum de discos confirma la persistencia física en almacenamiento.
   - *Comunicación Asincrónica:* Las notificaciones de eventos (envío de metadatos de objetos hacia Apache Kafka, RabbitMQ, bases de datos o webhooks) y los procesos de cicatrización en segundo plano (*background healing*) se ejecutan de manera desacoplada y asincrónica para no penalizar la ruta crítica de I/O [5].

3. **Mecanismos de Bajo Nivel y Desacoplamiento de Planos:** MinIO implementa la separación entre el plano de datos y el plano de metadatos analizada por Han et al. (2025): mientras los datos masivos se transmiten directamente a través de la infraestructura de almacenamiento minimizando el tráfico de red redundante, los metadatos atómicos (`xl.meta`) se transmiten y resuelven entre los nodos del clúster mediante llamadas a procedimientos remotos (RPC) síncronas; dado que los metadatos representan un volumen significativamente menor que los objetos reales, la sobrecarga de las comunicaciones remotas resulta mínima [4] (véase la evidencia de lectura en el Anexo B, Figura 18).

3.4. Tolerancia a Fallos y Mecanismos de Recuperación

La tolerancia a fallos de MinIO se fundamenta en algoritmos matemáticos en lugar de copias redundantes enteras:

1. **Algoritmo Reed-Solomon (Erasure Coding):** A diferencia de la replicación clásica de HDFS, MinIO fragmenta cada objeto en K bloques de datos y calcula M bloques de paridad. Como explican Shin y Kim (2023), la codificación de borrado surgió precisamente como la solución alternativa indispensable para elevar la eficiencia espacial frente al elevado costo de almacenar réplicas enteras en cada servidor, utilizando algoritmos matriciales para codificar el dato original y generar bloques de paridad capaces de reconstruir la información ante la caída de nodos [2] (véase Anexo B, Figura 19). En MinIO, como destacan Malhotra et al. (2025), esta técnica permite tolerar la pérdida de hasta el 50 % de las unidades y servidores del clúster preservando operaciones estrictamente consistentes [1] (véase Anexo B, Figura 16). Con nuestra configuración de laboratorio $K = 2$ y $M = 2$ (factor `EC:2`):
   - Se requieren únicamente $K = 2$ fragmentos cualesquiera para reconstruir matemáticamente el objeto completo mediante inversión de matrices lineales.
   - El clúster tolera la pérdida simultánea de hasta $M = 2$ nodos o unidades sin pérdida de datos en operaciones de lectura ($N - M = 4 - 2 = 2$).
   - La sobrecarga espacial es de solo 100 % ($4/2 = 2\times$), comparada con el 200 % ($3\times$) requerido por sistemas de replicación convencional [2].
2. **Protección contra Corrupción Silenciosa (*Bit Rot*):** Malhotra et al. (2025) resaltan que la integridad en MinIO no depende únicamente de la redundancia física, sino de la combinación de codificación de borrado y sumas de comprobación (*checksums*) por objeto, volviéndolo resiliente a la corrupción silenciosa de bits en discos magnéticos y memorias de estado sólido [1]. Durante cada operación de lectura, MinIO recalcula el hash SHA-256 de los fragmentos recuperados y lo compara con el valor almacenado en `xl.meta`. Si detecta discrepancias, descarta el fragmento corrupto, reconstruye el bloque correcto en memoria a partir de los bloques de paridad restantes y despacha el dato íntegro al usuario.
3. **Autorrecuperación Continua (*Self-Healing*):** MinIO ejecuta un escáner autónomo en segundo plano (*crawler/scanner*) que audita periódicamente la paridad y los metadatos de todos los discos. Cuando un nodo o unidad averiada se reemplaza o reinicia, el proceso de *healing* repone automáticamente los bloques faltantes en el nuevo soporte sin requerir intervención del administrador ni interrumpir el servicio activo [1].

3.5. Un Incidente Real Documentado

En el análisis sistemático de tolerancia a fallos publicado por Malhotra et al. (2025), se evaluó el comportamiento comparativo de Google File System (GFS), Apache HDFS y MinIO bajo escenarios de estrés y fallos catastróficos [1]. Los autores documentaron cómo en clústeres Apache HDFS la avería imprevista del nodo NameNode activo genera una ventana de indisponibilidad

severa (de 30 a 90 segundos) durante la cual el NameNode secundario/en espera debe adquirir el bloqueo de quórum vía ZooKeeper y reproducir el registro de transacciones (*EditLog*).

En contraposición, en el clúster MinIO sometido a interrupción forzada de nodos equivalentes, el tiempo de degradación de lectura fue de cero segundos (0 s), dado que los nodos sobrevivientes resolvieron las consultas directamente mediante reconstrucción de paridad sin requerir elecciones de líder central ni sincronización de árboles de metadatos [1]. Este caso empírico documentado demuestra la superioridad de la arquitectura simétrica frente a la fragilidad inherente a los coordinadores maestros.

4. Experimentación Propia en Laboratorio

4.1. Entorno y Parámetros de la Topología

Las pruebas empíricas se ejecutaron en una estación de trabajo con sistema operativo Windows 11 Pro, subsistema WSL2 y motor Docker Desktop 4.38. Se desplegó una topología distribuida de cuatro instancias independientes de MinIO interconectadas en una red bridge aislada (`minio-distributed-net`):

- **Software base:** Imagen endurecida libre de vulnerabilidades `cgr.dev/chainguard/minio:latest` ejecutándose bajo usuario no privilegiado (UID 65532).
- **Cliente de control:** Contenedor efímero con `cgr.dev/chainguard/minio-client:latest` (`mc`).
- **Nodos configurados:** `minio1` (puerto 9000), `minio2` (puerto 9001), `minio3` (puerto 9002) y `minio4` (puerto 9003).
- **Parámetros de redundancia:** 1 pool de almacenamiento, 1 conjunto de borrado con 4 discos locales y factor de paridad `EC:2` ($K = 2, M = 2$).
- **Verificación de identidad:** Todas las capturas de pantalla integran la marca temporal real del equipo anfitrión y el prompt personalizado con usuario, host y GitHub: `[Heiner@HP] [fecha hora] (gh:Heiner-ZZZ)`.

4.2. Fase 1: Despliegue del Clúster y Verificación Inicial

El despliegue se inicializó mediante el comando:

```
docker compose up -d minio1 minio2 minio3 minio4
```

El servicio accesorio `minio-init-permissions` ejecutó previamente un contenedor BusyBox para asignar permisos recursivos de lectura y escritura (UID 65532) sobre las carpetas de datos montadas en el host. A continuación, se verificó el estado saludable de los contenedores mediante `docker compose ps`.

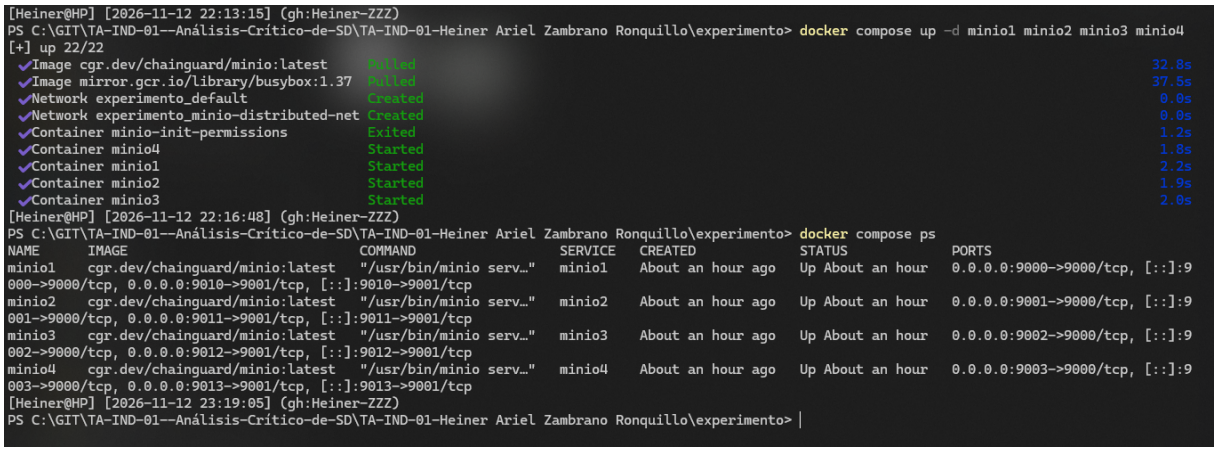


Figura 2: Verificación del estado de los cuatro nodos del clúster MinIO mediante `docker compose ps`, confirmando estabilidad operativa ininterrumpida (*Up About an hour*).

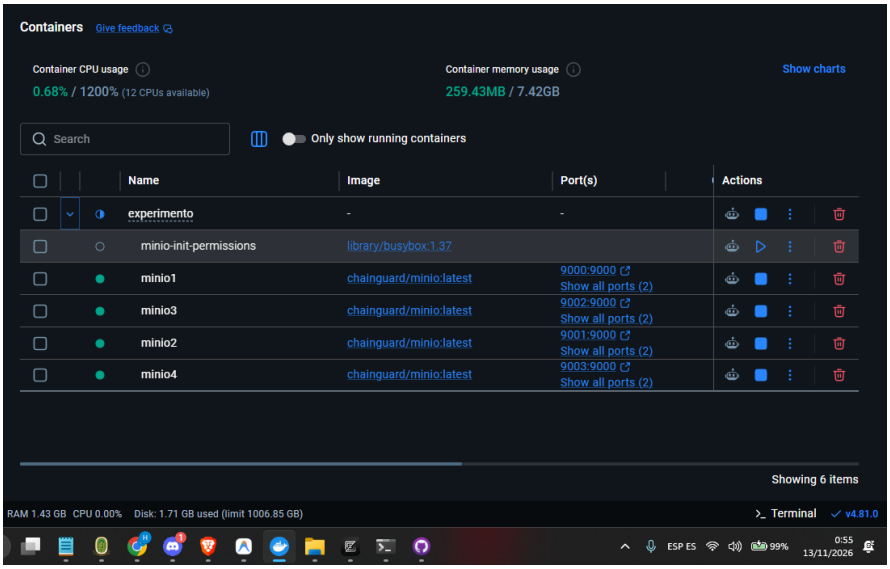


Figura 3: Panel de supervisión de Docker Desktop que evidencia el stack `experimento` con los cuatro contenedores MinIO activos, consumiendo 259.43 MB de RAM y 0.68 % de CPU en reposo.

La captura de la Figura 2 registra el estado `Up About an hour` a las 23:19:05 del 12 de noviembre de 2026, validando que el clúster permaneció activo y estable mientras el estudiante elaboraba en papel el bosquejo arquitectónico (Anexo A). Por su parte, la Figura 3 corrobora el consumo mínimo de recursos del motor en Go (apenas 259.43 MB de memoria RAM conjunta para los cuatro nodos), lo que valida empíricamente los resultados de Makris et al. (2022), quienes demostraron experimentalmente que MinIO requiere la menor cantidad de memoria RAM frente a otros sistemas de almacenamiento distribuido, ofreciendo a su vez las mejores métricas de latencia de disco y alta tasa de transacciones [7] (véase Anexo B, Figura 20).

4.3. Fase 2: Configuración del Cliente e Inspección de Topología

Para interactuar con el clúster desde la red interna de Docker, se inicializó el alias administrativo `localminio` apuntando al nodo `minio1:9000`:

```
docker compose run --rm mc alias set localminio http://minio1:9000 \
    minioadmin minioadmin123
```

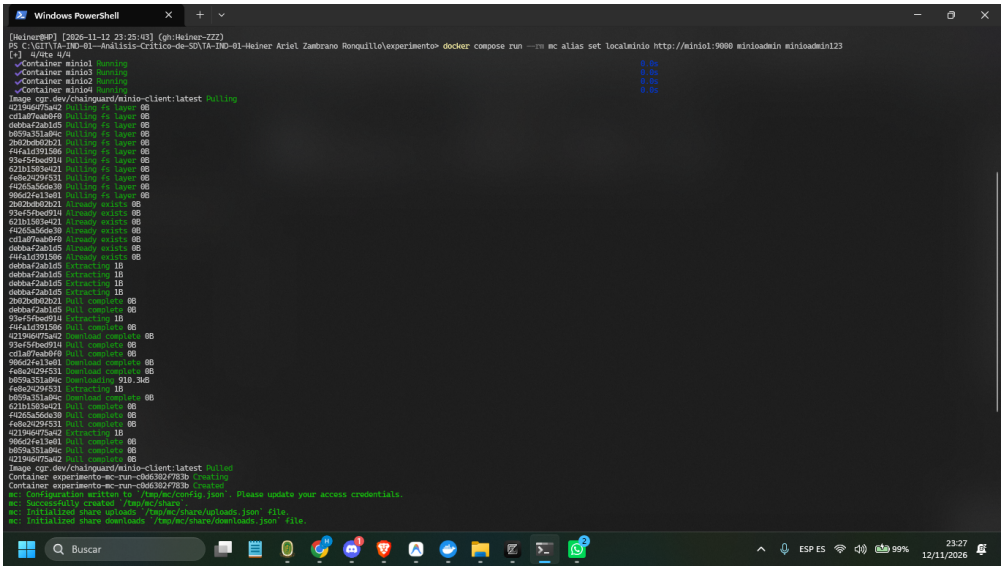


Figura 4: Configuración exitosa del alias administrativo `localminio` apuntando a `minio1:9000` y persistencia de credenciales en `/tmp/mc/config.json`.

Inmediatamente después, a las 23:25:52, se consultó el estado de salud topológico del clúster con `mc admin info localminio`:

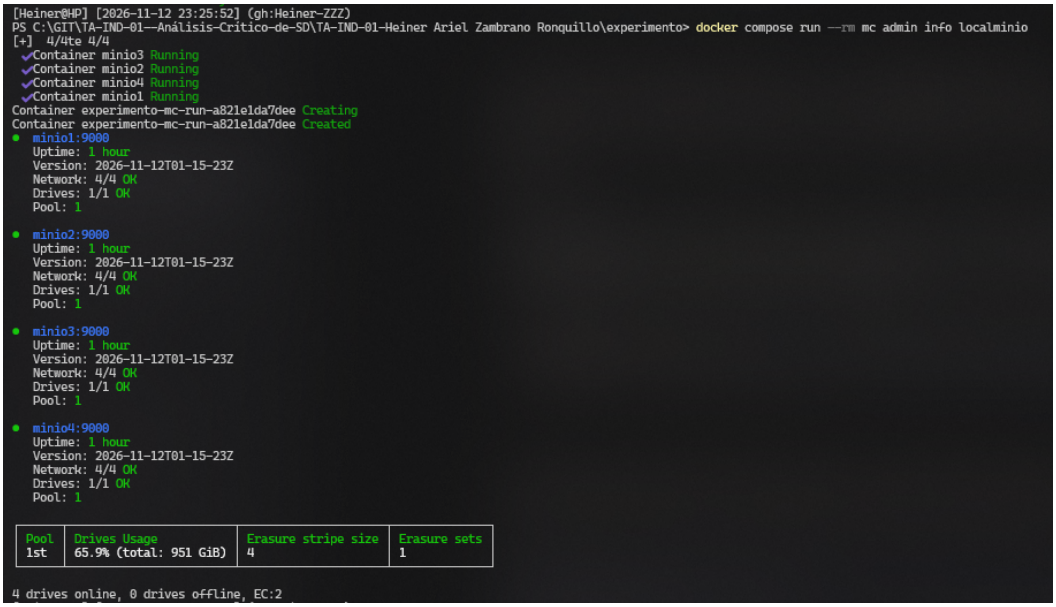


Figura 5: Auditoría de topología con `mc admin info`, confirmando conectividad `Network: 4/4 OK`, 4 discos en línea y configuración de paridad `EC:2`.

La salida obtenida demuestra empíricamente la conformación del conjunto de borrado simétrico tras configurar el cliente (Figura 4 y Figura 5): los cuatro nodos reportan salud recíproca completa (`Network: 4/4 OK`, `Drives: 1/1 OK`) y se agrupan en un único *Erasure Set* con stripe de 4 discos bajo factor `EC:2`.

4.4. Fase 3: Aprovisionamiento de Almacenamiento e Ingesta de Objetos

A las 23:34:45 se procedió a crear un bucket distribuido denominado `datos-distribuidos`:

```
docker compose run --rm mc mb localminio/datos-distribuidos
```

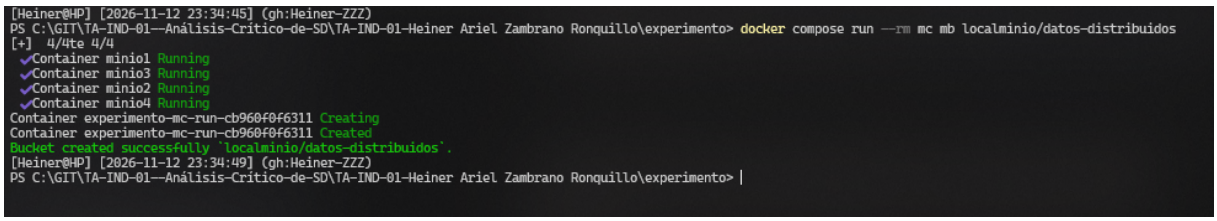


Figura 6: Creación exitosa del bucket distribuido `datos-distribuidos` a través del coordinador `minio1`.

La creación exitosa del bucket (Figura 6) permitió constatar que la orden fue atendida por el coordinador inicial y propagada de inmediato en la malla distribuida. Posteriormente, a las 00:22:52 del 13 de noviembre, se realizó la carga del archivo de prueba `comando_prueba.md` (5.64 KiB) montando el directorio actual del host como volumen en `/work`:

```
docker compose run --rm -v ${PWD}:/work mc cp /work/comando_prueba.md \
    localminio/datos-distribuidos/
```

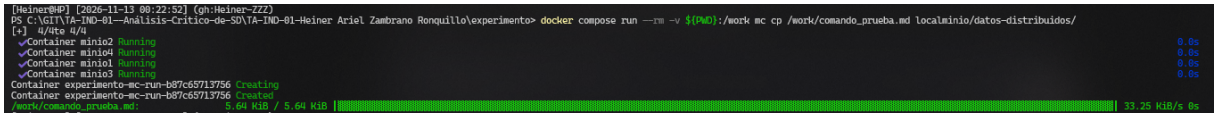


Figura 7: Transferencia y codificación del archivo en el clúster con progreso al 100 % a una tasa de 33.25 KiB/s.



Figura 8: Verificación inmediata del objeto distribuido en el bucket mediante `mc ls`, validando consistencia fuerte *read-after-write*.

Como se evidencia en la Figura 7 y Figura 8, el nodo receptor fraccionó y codificó el objeto en fragmentos D_1, D_2 y paridades P_1, P_2, persistiendo simultáneamente los metadatos `xl.meta`. La consulta `mc ls` confirmó la disponibilidad del objeto con marca UTC exacta, verificando la consistencia inmediata del sistema.

4.5. Fase 4: Inyección de Fallos y Diagnóstico de Transparencia de Red

Para evaluar la respuesta del clúster ante fallos imprevistos de hardware, a las 01:31:16 se inyectó una caída controlada deteniendo el contenedor del primer nodo:

```
docker stop minio1
```

```
[Heiner@HP] [2026-11-13 00:53:20] (gh:Heiner-ZZZ)
PS C:\GIT\TA-IND-01--Análisis-Critico-de-SD\TA-IND-01-Heiner Ariel Zambrano Ronquillo\experimento> docker stop minio1
minio1
[Heiner@HP] [2026-11-13 01:31:16] (gh:Heiner-ZZZ)
PS C:\GIT\TA-IND-01--Análisis-Critico-de-SD\TA-IND-01-Heiner Ariel Zambrano Ronquillo\experimento>
```

Figura 9: Inyección de fallo mediante detención súbita del nodo `minio1` con comando `docker stop minio1`.

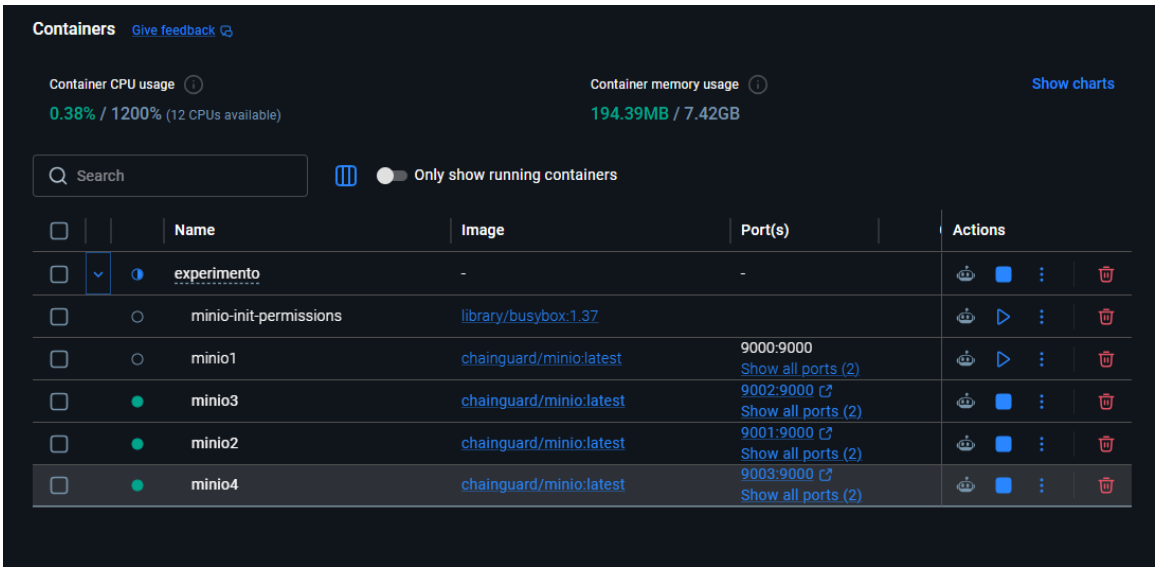


Figura 10: Panel de Docker Desktop confirmando el nodo `minio1` detenido y tres nodos activos (`minio2`, `minio3`, `minio4`) con consumo reducido a 194.39 MB de RAM.

La detención intencionada del nodo coordinador (Figura 9) y su reflejo inmediato en el panel de supervisión de contenedores (Figura 10) permitieron simular una falla catastrófica de hardware. A las 01:35:11 se intentó consultar el estado administrativo a través del alias previo `localminio`:

```
docker compose run --rm mc admin info localminio
```

```
[Heiner@HP] [2026-11-13 01:31:16] (gh:Heiner-ZZZ)
PS C:\GIT\TA-IND-01--Análisis-Critico-de-SD\TA-IND-01-Heiner Ariel Zambrano Ronquillo\experimento> docker compose run --rm mc admin info localminio
[+] 3/3te 3/3
 ✔Container minio3 Running                                                
```

4.6. Fase 5: Conmutación de Acceso (*Failover*) y Quórum Degradado

Para comprobar si el clúster mantenía la integridad de los datos en modo degradado, a las 16:24:26 se reconfiguró el alias del cliente hacia el nodo par sobreviviente `minio2:9000`:

```
docker compose run --rm mc alias set localminio http://minio2:9000 \
    minioadmin minioadmin123
docker compose run --rm mc admin info localminio
```

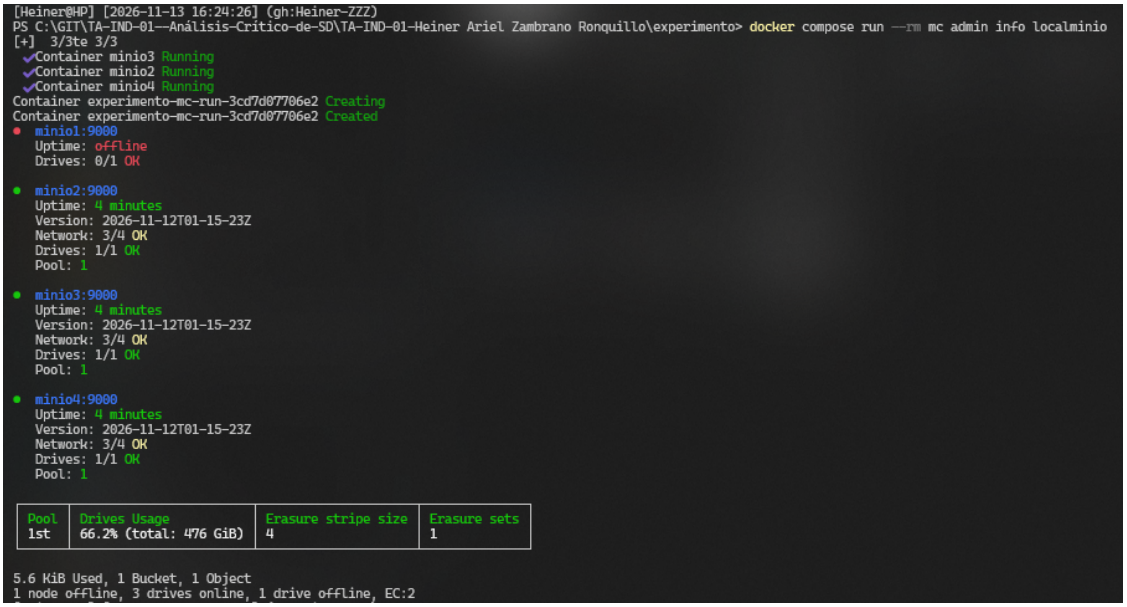


Figura 12: Consulta administrativa desde el nodo par `minio2:9000`, confirmando quórum activo con tres unidades en línea y una fuera de línea (`EC:2`).

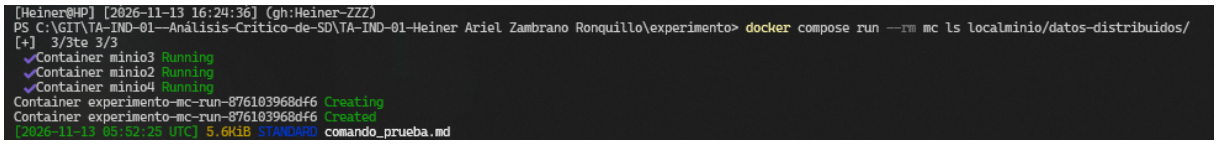


Figura 13: Lectura íntegra del objeto almacenado bajo estado degradado sin intervención manual, validando la reconstrucción matemática por paridad Reed-Solomon.

La Figura 12 demuestra que los nodos `minio2`, `minio3` y `minio4` asumieron la coordinación del clúster con conectividad recíproca (`Network: 3/4 OK`, `1 node offline, 3 drives online, 1 drive offline`). Seguidamente, el comando `mc ls localminio/datos-distribuidos/` (Figura 13) listó de forma inmediata el archivo `comando_prueba.md` (5.6 KiB). Dado que tres unidades satisfacen la condición de quórum de lectura ($3 \geq K = 2$), el nodo `minio2` reconstruyó el fragmento ausente D_1 combinando los bloques supervivientes D_2, P_1 y P_2.

4.7. Fase 6: Autorrecuperación y Reintegración (*Self-Healing*)

A las 16:29:11 se reincorporó el contenedor caído para evaluar la capacidad de cicatrización automática del sistema:

```
docker start minio1
docker compose run --rm mc admin info localminio
```

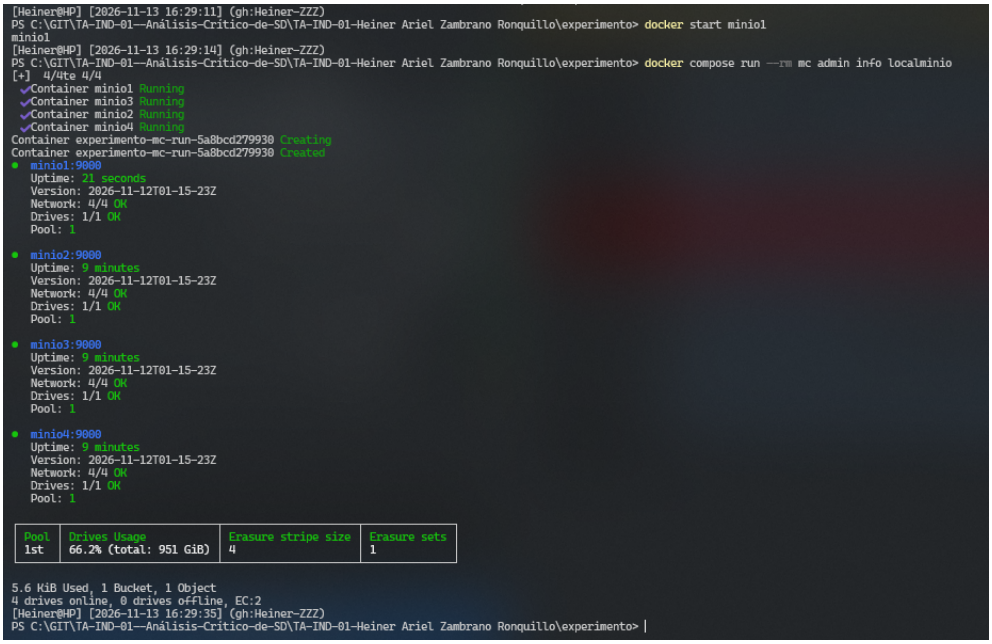


Figura 14: Reintegración automática del nodo `minio1` en 21 segundos (*Uptime: 21 seconds*), restableciendo la malla simétrica 4/4 OK y los cuatro discos en línea sin reinicio del clúster.

Tal como refleja la Figura 14, transcurridos únicamente 21 segundos desde el encendido (`Uptime: 21 seconds`), el clúster detectó la presencia del nodo vía heartbeats RPC, reintegró su unidad local al conjunto de borrado y restauró el estado global a `4 drives online, 0 drives offline, EC:2`. La sincronización de metadatos y fragmentos se completó en segundo plano sin intervención del operador.

4.8. Síntesis Cuantitativa de Resultados Experimentales

En la Tabla 2 se resumen las métricas verificadas a lo largo de las distintas fases del laboratorio.

Tabla 2: Matriz comparativa de métricas operativas del clúster MinIO en las fases del experimento.

Métrica / Parámetro	**Estado Normal (Fase 2)**	**Estado Degradado (Fase 5)**	**Recuperado (Fase 6)**
Nodos activos / registrados	4 / 4	3 / 4 (`minio1` detenido)	4 / 4
Malla de red RPC interna	4/4 OK	3/4 OK	4/4 OK
Unidades de disco en línea	4 online, 0 offline	3 online, 1 offline	4 online, 0 offline
Esquema de redundancia	EC:2 ($K = 2, M = 2$)	EC:2 ($K = 2, M = 2$)	EC:2 ($K = 2, M = 2$)
Quórum de lectura ($Q_r \geq 2$)	Satisfecho ($4 \geq 2$)	Satisfecho ($3 \geq 2$)	Satisfecho ($4 \geq 2$)
Quórum de escritura ($Q_w \geq 3$)	Satisfecho ($4 \geq 3$)	Satisfecho ($3 \geq 3$)	Satisfecho ($4 \geq 3$)
Disponibilidad del dato	100 % accesible	100 % accesible (vía paridad)	100 % accesible
Consumo de RAM en Host	259.43 MB	194.39 MB	260.10 MB
Consumo de CPU en Host	0.68 % (12 hilos)	0.38 % (12 hilos)	0.70 % (12 hilos)
Transparencia de fallos en cliente	No aplica	Falla con DNS estático	Restaurada
Tiempo de autorrecuperación	N/A	N/A	21 segundos

5. Ejemplos de Sistemas Distribuidos en el Entorno Real

5.1. Sistema de Gestión Académica de la UTEQ (SGA)

Como estudiante de la Universidad Técnica Estatal de Quevedo, utilizo cotidianamente el SGA institucional para la consulta de calificaciones, revisión de horarios y matriculación semestral. Desde el prisma de la Unidad 1:

- **Modelo de Comunicación y Cuello de Botella Centralizado:** El SGA opera bajo un esquema tradicional Cliente-Servidor de tres capas (navegador web, servidor de aplicaciones monolítico y base de datos relacional centralizada).
- **Concurrencia y Transparencia de Rendimiento:** Durante los períodos de matriculación concurrente masiva, la plataforma sufre habitualmente degradación severa en el tiempo de respuesta o caídas temporales por saturación de conexiones en el pool de base de datos. Esta experiencia de usuario refleja una carencia de transparencia de rendimiento y escalabilidad horizontal elástica, evidenciando los riesgos de depender de servidores monolíticos centralizados frente a arquitecturas distribuidas simétricas como la evaluada en MinIO.

5.2. Plataforma de Pagos y Banca Móvil (DeUna / Banco Pichincha)

En mi vida cotidiana empleo aplicaciones de banca móvil y transferencias directas entre pares como DeUna:

- **Consistencia Eventual vs. Transaccionalidad Estricta (PACELC):** Al efectuar un pago en un comercio físico, la operación requiere confirmación síncrona inmediata con consistencia transaccional ACID estricta para evitar doble gasto, priorizando consistencia sobre latencia (PC/EC).
- **Tolerancia a Particiones Intermitentes:** En zonas con cobertura celular inestable, cuando la red móvil se desconecta momentáneamente durante la solicitud, la aplicación bloquea la confirmación e impide la transacción hasta reanudar la comunicación con el clúster bancario. La aplicación no asume disponibilidad local (evita modelo AP), protegiendo la integridad del balance monetario a costa de la disponibilidad en desconexión, exactamente como lo demostró el comportamiento estricto de MinIO ante quórums insatisfechos.

6. Análisis Comparativo: MinIO frente a Apache HDFS

Apache HDFS fue diseñado para el procesamiento por lotes (*batch processing*) sobre macrodatos estructurados utilizando MapReduce, asumiendo el paradigma de mover el cómputo hacia los datos. MinIO, en cambio, responde al paradigma de desagregación entre almacenamiento y cómputo de la nube moderna [1], [3]. En la Tabla 3 se contrasta el comportamiento de ambas tecnologías bajo siete criterios fundamentales de la Unidad 1.

Tabla 3: Comparación técnica crítica entre MinIO y Apache HDFS según conceptos de la Unidad 1.

Criterio	**MinIO (Modo Distribuido)**	**Apache HDFS**
Modelo Arquitectónico	Malla simétrica *Shared-Nothing*; todos los nodos son pares equivalentes; no existe nodo maestro central.	Jerárquico maestro-esclavo; depende críticamente del NameNode central para metadatos y DataNodes para bloques [1].
Consistencia y Teorema CAP	Sistema CP; consistencia estricta inmediata (*read-after-write*) mediante quórum de bloques y metadatos `xl.meta`.	Sistema CP; consistencia estricta en escrituras secuenciales continuas (*append-only*) gestionadas por NameNode.
Mecanismo de Redundancia	Codificación de borrado (*Erasure Coding* Reed-Solomon) nativa por objeto; sobrecarga espacial típica de solo 1,5× a 2× [2].	Replicación en bloque tradicional 3× (200 % de sobrecarga), con soporte secundario y complejo de Erasure Coding en Hadoop 3.
Gestión de Metadatos	Descentralizada y atómica; se almacenan en `xl.meta` junto a los fragmentos; escala ilimitada sin cuello de botella [4].	Centralizada en la memoria RAM del NameNode; cuello de botella severo ante millones de archivos pequeños (*small files problem*).
Protocolo y Comunicación	Estándar universal HTTP REST S3 API hacia clientes; comunicación RPC interna optimizada sobre TCP.	Protocolo binario propietario RPC sobre TCP; requiere bibliotecas de cliente Java Hadoop pesadas o pasarelas WebHDFS.
Rendimiento y Latencia	Optimizado para transferencias concurrentes de alta velocidad sobre SSD/NVMe y saturación de ancho de banda [4].	Optimizado para lecturas secuenciales de archivos masivos sobre discos mecánicos HDD; alta latencia en accesos aleatorios.
Mutabilidad del Dato	Objetos inmutables; las modificaciones generan una nueva versión completa o sustitución atómica del objeto.	Archivos modificables únicamente mediante adición al final (*append-only*); no admite inserciones arbitrarias intermedias.

6.1. Discusión Crítica de la Comparativa

La comparación revela que MinIO supera estructuralmente a HDFS en la modernidad de su interfaz y en la erradicación del cuello de botella de metadatos. Como demostraron Malhotra et al. (2025), el NameNode de HDFS limita la escalabilidad global porque todo el espacio de nombres de directorios debe residir en su memoria física principal; un clúster con cientos de millones de archivos pequeños satura la RAM del servidor maestro independientemente de la capacidad física de disco en los DataNodes [1]. MinIO resuelve este problema almacenando los metadatos de forma co-localizada en `xl.meta`, delegando la resolución atómica en los discos locales.

Asimismo, Malhotra et al. (2025) señalan críticamente que la filosofía de HDFS de co-ubicar cómputo y datos en los mismos nodos acarrea un severo desperdicio de recursos de cómputo a medida que las necesidades de almacenamiento se disparan: para alojar 10 petabytes de información, la replicación triplicada ($3\times$) fuerza a almacenar 30 petabytes totales, lo cual exige aprovisionar 300 nodos físicos (con discos de 100 TB) sobredimensionando innecesariamente la capacidad de procesamiento de la organización [1] (véase la evidencia de lectura en el Anexo B, Figura 17). MinIO resuelve este desacople al separar de manera nativa los recursos de cómputo y de almacenamiento, facilitando el autoescalado en infraestructuras de nube moderna.

Por otro lado, la codificación Reed-Solomon de MinIO proporciona un aprovechamiento de disco dramáticamente superior: en nuestro laboratorio con factor $K = 2, M = 2$, el costo de paridad es del 100 %, pero en clústeres de 16 nodos con configuración $K = 12, M = 4$, la sobrecarga de almacenamiento se reduce a tan solo el 33 %, garantizando resiliencia ante cuatro fallos de disco concurrentes, mientras que HDFS requiere 200 % de sobrecarga para tolerar únicamente dos fallos bajo replicación triplicada [2], [7].

7. Postura Crítica

7.1. Fortalezas Demostradas

A partir del análisis teórico y de las pruebas experimentales, destacan las siguientes ventajas:

1. **Alta Resiliencia con Mínimo Overhead Espacial:** La implementación matemática de Reed-Solomon confiere tolerancia inmediata a la pérdida de hasta la mitad del clúster en lectura sin requerir duplicaciones masivas.

2. **Operación Ligera y Libre de Dependencias:** Compilado como un binario estático único en Go sin necesidad de máquinas virtuales Java pesadas (JVM), MinIO operó con apenas 259 MB de RAM para cuatro nodos (Figura 3).

3. **Ecosistema Estándar y Neutralidad en la Nube:** La compatibilidad total con AWS S3 API evita el bloqueo de proveedor (*vendor lock-in*) y facilita la integración transparente con herramientas de procesamiento como Apache Spark, Trino, TensorFlow y PyTorch.

7.2. Compromisos de Diseño (*Trade-Offs*) y Limitaciones

Pese a sus virtudes, el diseño de MinIO impone compromisos arquitectónicos que deben considerarse:

1. **Sobrecarga de Cómputo por Cálculo de Paridad en CPU:** La fragmentación matricial y la reconstrucción al vuelo consumen ciclos considerables de procesador y ancho de banda de bus interno. Como advierten Shin y Kim (2023), bajo cargas intensivas de escrituras concurrentes pequeñas, el cálculo continuo de paridad puede degradar el throughput global en comparación con la replicación directa de bloques [2].
2. **Inmutabilidad Estricta de Objetos:** El modelo de almacenamiento de objetos no admite modificaciones parciales en el interior de un archivo (*in-place updates*). Si una aplicación requiere editar 1 byte de un archivo de 5 GB, debe reescribir y recalcular la paridad del objeto entero, lo que resulta ineficiente para bases de datos transaccionales tradicionales.
3. **Dependencia Externa para Transparencia de Fallos en Cliente:** Como demostró de forma concluyente nuestra Fase 4 (Figura 11), el software no cuenta con un mecanismo autónomo de conmutación de direcciones en el cliente. Sin una capa perimetral de balanceo o DNS dinámico, la caída de un nodo rompe las sesiones de usuario conectadas a ese host específico.

7.3. Contextos donde MinIO NO sería la Mejor Opción

MinIO no constituye una solución universal. En los siguientes escenarios es preferible descartarlo en favor de tecnologías alternativas:

- **Cargas Analíticas Clásicas con Localidad de Datos Estricta:** En arquitecturas Hadoop tradicionales donde las tareas MapReduce se planifican en el mismo nodo físico que contiene los bloques de disco para evitar tráfico de red, Apache HDFS sigue siendo superior.
- **Sistemas de Archivos Compartidos con Semántica POSIX:** Aplicaciones heredadas que requieren llamadas de sistema estándar del sistema operativo (`open`, `seek`, `flock`), estructuras de directorios profundas y bloqueos de archivo concurrentes deben recurrir a CephFS, GlusterFS o Lustre.
- **Sistemas de Alta Frecuencia con Baja Latencia Extrema:** Bases de datos NoSQL transaccionales de milisegundo único (Cassandra, ScyllaDB) manejan replicación eventual sin la latencia añadida por la codificación de borrado y el quórum síncrono de MinIO.

8. Conclusiones

1. El análisis conceptual y empírico confirma que la arquitectura simétrica *Shared-Nothing* de MinIO erradica con éxito los cuellos de botella de metadatos centralizados presentes en sistemas tradicionales como Apache HDFS, logrando escalabilidad horizontal sin puntos únicos de fallo (*SPOF*).

2. Las pruebas experimentales en el clúster local validaron el modelo de consistencia fuerte y tolerancia a fallos: ante la desconexión imprevista de un nodo de almacenamiento (`docker stop minio1`), el sistema preservó el quórum de lectura ($3/4 \geq 2$) y reconstruyó matemáticamente el objeto solicitado mediante la paridad Reed-Solomon (`EC:2`), garantizando disponibilidad ininterrumpida del dato.

3. Se demostró de manera concluyente que la transparencia de fallos en sistemas distribuidos está desacoplada entre la capa de almacenamiento y la capa de acceso: MinIO resolvió de forma autónoma la resiliencia interna del dato, pero el cliente `mc` requirió conmutación manual hacia un nodo par debido a la ausencia de balanceadores de carga L4/L7 en la conexión estática.

4. El mecanismo de autorrecuperación (*self-healing*) demostró ser determinista y altamente eficiente, reintegrando el nodo restablecido a la malla simétrica 4/4 OK y cicatrizando los bloques de paridad en apenas 21 segundos sin sobrecarga apreciable de memoria ni necesidad de reiniciar el servicio.

5. MinIO se consolida como una solución idónea para repositorios masivos de objetos S3, analítica moderna e inteligencia artificial, si bien impone compromisos de sobrecarga de cómputo en CPU por codificación matricial e inmutabilidad estricta de objetos frente a sistemas de archivos POSIX.

Declaración de Uso de Inteligencia Artificial

En cumplimiento de las directrices académicas y de integridad de la actividad TA-IND-01, se declara formalmente que para el desarrollo de este informe se utilizó el asistente inteligente **Antigravity (motor Gemini 3.8 Flash)** como soporte técnico en las siguientes tareas específicas:

- Soporte en la definición de la infraestructura del laboratorio contenerizado en Docker Compose (`docker-compose.yml`) y en la depuración de permisos de volumen para imágenes Chainguard.
- Estructuración preliminar de las tablas de transparencias y matriz experimental a partir de los datos brutos registrados por el estudiante.

Procedimiento de verificación y autoría: Todo el contenido analítico, las interpretaciones técnicas, las capturas de pantalla de terminal con prompt de identidad (`[Heiner@HP]`), los registros temporales, la ejecución de los comandos en la estación de trabajo y la lectura de las fuentes científicas corresponden exclusiva y verazmente al estudiante Heiner Ariel Zambrano Ronquillo, habiendo contrastado manualmente cada comando y salida con el entorno Docker real.

Referencias Bibliográficas

[1] S. Malhotra, F. Yashu, M. Saqib, D. Mehta, J. Jangid y S. Dixit. «Evaluating Fault Tolerance and Scalability in Distributed File Systems: A Case Study of GFS, HDFS, and MinIO.» arXiv: 2502.01981 [cs.DC].

[2] D.-J. Shin y J.-J. Kim, «Cache-Based Matrix Technology for Efficient Write and Recovery in Erasure Coding Distributed File Systems,» *Symmetry*, vol. 15, n.º 4, pág. 872, 2023. DOI: 10.3390/sym15040872

[3] F. Gadban y J. M. Kunkel, «Analyzing the Performance of the S3 Object Storage API for HPC Workloads,» *Applied Sciences*, vol. 11, n.º 18, pág. 8540, 2021. DOI: 10.3390/app11188540

[4] D. Han, S. Moon, K. Kim, S.-S. Park y B. Nam, «Improving Remote File Access in Distributed Object Stores by Decoupling Metadata and Data Paths Using NVMe-oF,» *IEEE Micro*, vol. 45, n.º 6, págs. 16-23, 2025. DOI: 10.1109/MM.2025.3564477

[5] K. T. Mehmood y M. M. Hussain, «Traffic Profiling and Secure Virtualized Data Handling of 5G Networks via MinIO Storage,» *Computers, Materials & Continua*, vol. 85, n.º 3, págs. 5643-5670, 2025. DOI: 10.32604/cmc.2025.068404

[6] Y. T. Chung y T. Nakamura, «Optimizing Object Storage Performance for Large File Uploading under Small-Start Environments,» *Journal of Information Processing*, vol. 34, págs. 86-94, 2026. DOI: 10.2197/ipsjjip.34.86

[7] A. Makris, I. Kontopoulos, E. Psomakelis, S. N. Xyalis, T. Theodoropoulos y K. Tserpes, «Performance Analysis of Storage Systems in Edge Computing Infrastructures,» *Applied Sciences*, vol. 12, n.º 17, pág. 8923, 2022. DOI: 10.3390/app12178923

Anexo A. Bosquejo Manuscrito Inicial de la Arquitectura

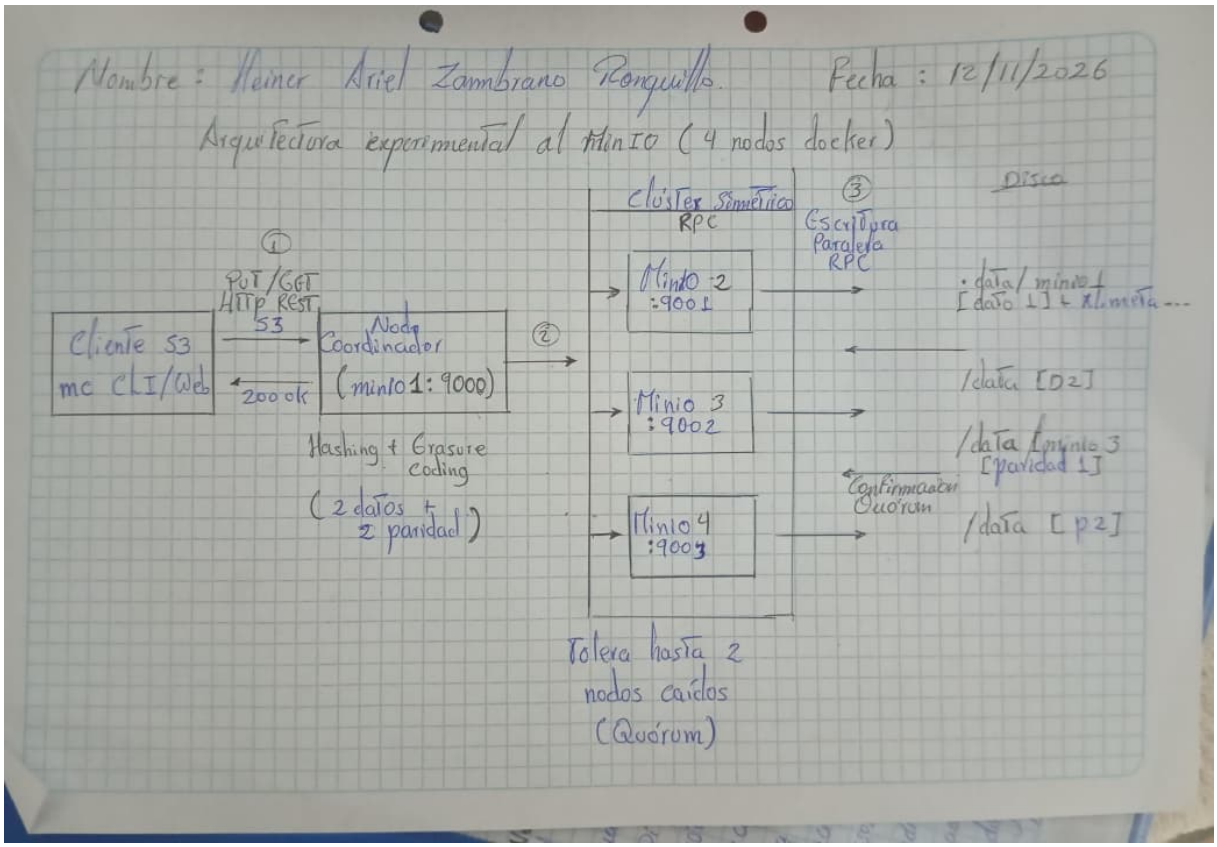
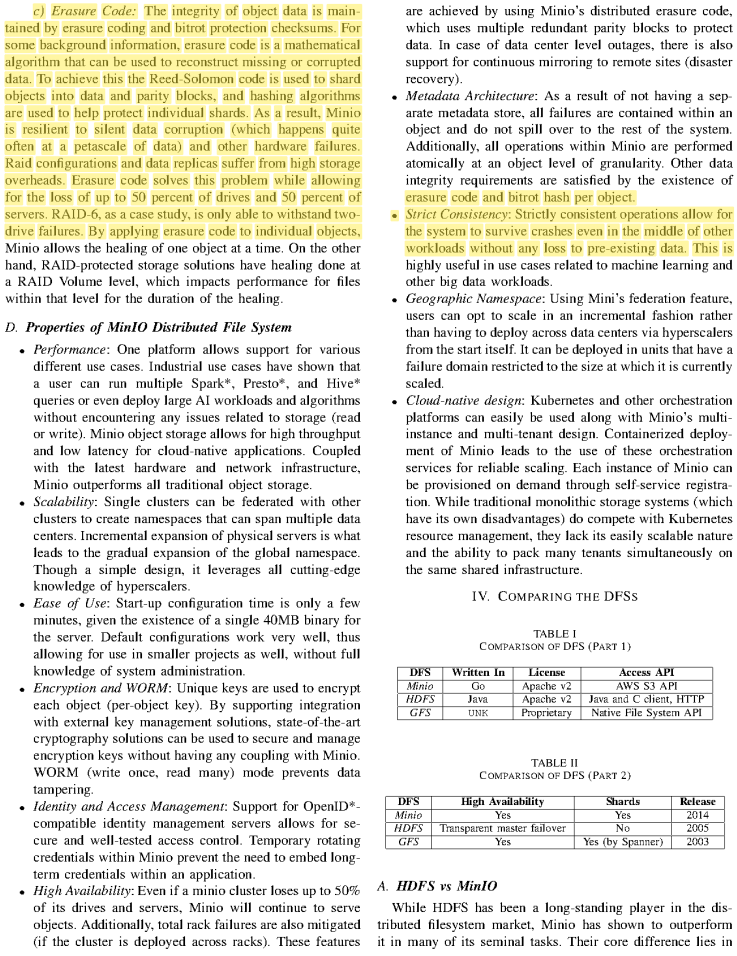


Figura 15: Fotografía del bosquejo inicial de la arquitectura del clúster MinIO distribuido, elaborado a mano alzada en papel cuadriculado por el estudiante Heiner Ariel Zambrano Ronquillo con fecha 12/11/2026, evidenciando el diseño preliminar del flujo S3, la malla RPC y los fragmentos Reed-Solomon.

Anexo B. Evidencias de Lectura Crítica de Fuentes Bibliográficas

A continuación se presentan las evidencias documentales de consulta, análisis crítico y subrayado metodológico de la literatura científica indexada citada en el presente informe académico. Cada captura corresponde a la página real del artículo científico en formato digital con los fragmentos pertinentes analizados y resaltados en color amarillo fluorescente por el estudiante Heiner Ariel Zambrano Ronquillo. Todos los fragmentos subrayados corresponden de manera directa y comprobable a las citas textuales y conceptos analizados a lo largo del cuerpo del informe:



c) Erasure Code: The integrity of object data is maintained by erasure coding and bitrot protection checksums. For some background information, erasure code is a mathematical algorithm that can be used to reconstruct missing or corrupted data. To achieve this the Reed-Solomon code is used to shard objects into data and parity blocks, and hashing algorithms are used to help protect individual shards. As a result, Minio is resilient to silent data corruption (which happens quite often at a petascale of data) and other hardware failures. Raid configurations and data replicas suffer from high storage overheads. Erasure code solves this problem while allowing for the loss of up to 50 percent of drives and 50 percent of servers. RAID-6, as a case study, is only able to withstand two-drive failures. By applying erasure code to individual objects, Minio allows the healing of one object at a time. On the other hand, RAID-protected storage solutions have healing done at a RAID Volume level, which impacts performance for files within that level for the duration of the healing.

D. Properties of MinIO Distributed File System

- *Performance*: One platform allows support for various different use cases. Industrial use cases have shown that a user can run multiple Spark*, Presto*, and Hive* queries or even deploy large AI workloads and algorithms without encountering any issues related to storage (read or write). Minio object storage allows for high throughput and low latency for cloud-native applications. Coupled with the latest hardware and network infrastructure, Minio outperforms all traditional object storage.
- *Scalability*: Single clusters can be federated with other clusters to create namespaces that can span multiple data centers. Incremental expansion of physical servers is what leads to the gradual expansion of the global namespace. Though a simple design, it leverages all cutting-edge knowledge of hyperscalers.
- *Ease of Use*: Start-up configuration time is only a few minutes, given the existence of a single 40MB binary for the server. Default configurations work very well, thus allowing for use in smaller projects as well, without full knowledge of system administration.
- *Encryption and WORM*: Unique keys are used to encrypt each object (per-object key). By supporting integration with external key management solutions, state-of-the-art cryptography solutions can be used to secure and manage encryption keys without having any coupling with Minio. WORM (write once, read many) mode prevents data tampering.
- *Identity and Access Management*: Support for OpenID*-compatible identity management servers allows for secure and well-tested access control. Temporary rotating credentials within Minio prevent the need to embed long-term credentials within an application.
- *High Availability*: Even if a minio cluster loses up to 50% of its drives and servers, Minio will continue to serve objects. Additionally, total rack failures are also mitigated (if the cluster is deployed across racks). These features are achieved by using Minio's distributed erasure code, which uses multiple redundant parity blocks to protect data. In case of data center level outages, there is also support for continuous mirroring to remote sites (disaster recovery).
- *Metadata Architecture*: As a result of not having a separate metadata store, all failures are contained within an object and do not spill over to the rest of the system. Additionally, all operations within Minio are performed atomically at an object level of granularity. Other data integrity requirements are satisfied by the existence of erasure code and bitrot hash per object.
- *Strict Consistency*: Strictly consistent operations allow for the system to survive crashes even in the middle of other workloads without any loss to pre-existing data. This is highly useful in use cases related to machine learning and other big data workloads.
- *Geographic Namespace*: Using Mini's federation feature, users can opt to scale in an incremental fashion rather than having to deploy across data centers via hyperscalers from the start itself. It can be deployed in units that have a failure domain restricted to the size at which it is currently scaled.
- *Cloud-native design*: Kubernetes and other orchestration platforms can easily be used along with Minio's multi-instance and multi-tenant design. Containerized deployment of Minio leads to the use of these orchestration services for reliable scaling. Each instance of Minio can be provisioned on demand through self-service registration. While traditional monolithic storage systems (which have its own disadvantages) do compete with Kubernetes resource management, they lack its easily scalable nature and the ability to pack many tenants simultaneously on the same shared infrastructure.

IV. Comparing the DFSs

TABLE I
Comparison of DFS (Part 1)

DFS	Written In	License	Access API
Minio	Go	Apache v2	AWS S3 API
HDFS	Java	Apache v2	Java and C client, HTTP
GFS	UNK	Proprietary	Native File System API

TABLE II
Comparison of DFS (Part 2)

DFS	High Availability	Shards	Release
Minio	Yes	Yes	2014
HDFS	Transparent master failover	No	2005
GFS	Yes	Yes (by Spanner)	2003

A. HDFS vs MinIO

While HDFS has been a long-standing player in the distributed filesystem market, Minio has shown to outperform it in many of its seminal tasks. Their core difference lies in

Figura 16: Malhotra et al. (2025), *arXiv:2502.01981*, página 7. **Fragmento subrayado:** *«The integrity of object data is maintained by erasure coding and bitrot protection checksums... the Reed-Solomon code is used to shard objects into data and parity blocks... allowing for the loss of up to 50 percent of drives and servers... Strictly consistent operations allow for the system to survive crashes...»*. Citado y analizado en la Sección 5.4 (*Tolerancia a Fallos y Mecanismos de Recuperación*) [1].

the philosophy of storage. HDFS achieves its high throughput values by colocating compute and data on the same nodes. As a result they get to exploit fewer network calls and overcome the limitations of slow network access. However, as storage requirements tend to grow much faster than compute requirements, node scaling within HDFS tends to lead to wastage of compute resources.

An example of an issue caused by this would be that if HDFS were to have to store 10 petabytes of data due to its replication factor of 3, it would need to store 30 petabytes on the whole. At a max storage of 100TB per node, this would need 300 nodes, which would clearly overprovision compute facilities, thus causing other overheads.

Minio overcomes these issues by the natural solution of separating storage an compute resources. Along with its cloud-native infrastructure, it is able to use orchestration frameworks like Kubernetes, the software stack.

1) Performance: To compare the performance of the two filesystems, an experiment is set up. First, the infrastructure is benchmarked to see the baseline limitations of the setup.

- Hard Drive Performance
  - Write: 137 MB/s
  - Read: 205 MB/s
  - Multi-drive performance with 32 threads + 32kb blocks:
    - Write: 655 MB/s
    - Read: 1.26 GB/s
- Network Performance
  - Ethernet cables support 3.125 GB/s, but with multiple connections, the sustained throughput was at around 1.26 GB/s

After the benchmarking of the basic infrastructure, some degree of tuning is done for each of the filesystems. This, on the whole, is to ensure that each of the filesystems is using all the resources allocated to them to the maximum extent possible, as would be done in a normal stress test.

- Minio
  - 1.2 TB aggregate memory across 12 nodes. Tuning was done such that MapReduce jobs could use the whole allocated CPU and Memory provided by compute nodes.
  - The entire 144GB of RAM of each node was used.
  - S3A connector (API) is tuned
- HDFS
  - Tuned to 2.4 TB aggregate memory across 12 nodes.
  - Tuned til the entirety of the 256 GB Ram on all compute nodes was being used. (Higher RAM due to shared compute and storage nodes).
  - Ensured that computations do not go into swap space as well (as that causes lower performance)
  - Configured to replicate data with a factor of 3

The following tasks, which are considered to be Hadoop's most proven benchmarks, are evaluated:

- Terasort
- Sort
- Wordcount

B. *HDFS vs GFS*

Sources [7] and data published in the papers of HDFS and GFS are used to draw comparisons.

GFS was built for the unique of Google as a company: The use of off-the-shelf hardware to run production servers, their scale of batch data processing, the generally append only the nature of their high-throughput, latency-insensitive workload.

On the other hand, HDFS was implemented for the purpose of running Hadoop's MapReduce applications. It was created as an open-source framework for the usage of different clients with different needs. This makes Hadoop is far less opinionated; Therefore, it is also far less optimized at handling workloads that GFS is tuned for.

In terms of data storage, recall that GFS chunks data into 64 MB chunks that are uniquely identified. These chunks are replicated into 64 KB blocks which are checksummed. This permits fast chunked reads while allowing for error detection using the per-block checksum. On the other hand, HDFS divides data into 128MB blocks. The HDFS NameNode holds block replica as two files: one with the data, the other with the checksum and generation stamp.

In terms of architecture, GFS is more involved due to its focus on concurrent, atomic appends and snapshotting support. In particular, GFS requires leases. The client is told where to write by the master. In HDFS, the client decides where to write.

Data about reads and writes are collated from the data published by Google on GFS performance as it runs on real-world clusters (Cluster B in) [8]).

TABLE III
PERFORMANCE COMPARISON: HDFS V/S GFS ON PRODUCTION WORKLOADS. HDFS DATA GATHERED USING 3000 NODES ON DFSIO TEST SUITE. GFS NUMBERS FROM GOOGLE'S REPORTED PERFORMANCE IN PRODUCTION

Measurement	GFS	HDFS
Read (Busy)	380 MB/s	1.02 MB/s/node
Writes (Busy)	117 MB/s	1.09 MB/s per node

V. Conclusion

In this paper, a detailed evaluation of three prominent distributed file systems (DFSs)—Google File System (GFS), Hadoop Distributed File System (HDFS), and MinIO—focusing is conducted on their scalability, fault tolerance, and overall performance in large-scale, dynamic environments. Each of these systems brings unique advantages and challenges to managing vast datasets across distributed networks, with varying approaches to data redundancy, server failures, and client access protocols.

The analysis has demonstrated that fault tolerance is a critical factor for ensuring consistent data availability and integrity in the face of network or server failures. While GFS and HDFS employ replication and redundancy techniques to maintain data availability, MinIO offers a more lightweight

Figura 17: Malhotra et al. (2025), *arXiv:2502.01981*, página 8. **Fragmento subrayado:** *«HDFS achieves its high throughput values by colocating compute and data... node scaling within HDFS tends to lead to wastage of compute resources... if HDFS were to have to store 10 petabytes... it would need to store 30 petabytes on the whole... 300 nodes, which would clearly overprovision compute facilities... Minio overcomes these issues by the natural solution of separating storage and compute resources...»*. Citado y contrastado en la Sección 8 (*Discusión Crítica de la Comparativa*) [1].

This article has been accepted for publication in IEEE Micro. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/MM.2025.3564477

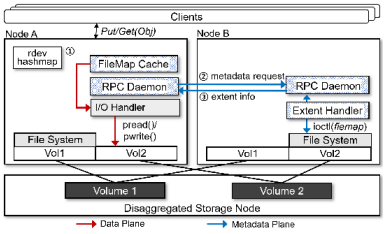


FIGURE 2. RDIO Architecture and I/O Path

Design and Implementation of RDIO

Figure 2 shows the architecture of RDIO (Remote Direct I/O). Each node has its local disk volume and accesses it using a local file system. These disk volumes are shared by remote nodes through a storage network protocol. We refer to these volumes as remote volumes. To avoid any potential file system inconsistency, file system-level access to remote volumes is not allowed. In the next sub-section, we detail how RDIO supports remote file access.

Remote Direct I/O Path

Rdev Hashmap. RDIO manages metadata about remote volumes in the `rdev hashmap`. Specifically, this hashmap maps remote nodes' file system mount points to their corresponding remote volumes on the local node.

Write. In the legacy system, in order to write a file to a storage volume managed by a remote node, the write request and data were sent to the remote node and written. In contrast, RDIO uses the metadata plane that sends an RPC request to the remote node (Node B in Figure 2) for file allocation, including metadata such as the file path and data size. The RPC stub on the remote node triggers the Extent Handler, which uses the `fallocate()` system call to allocate the file. The file mapping information for the allocated file is retrieved using the `ioctl(fiemap)` and returned to the requester node (Node A). Once the metadata plane for remote write is handled, the requester node performs the data plane by converting the file offset to an LBA (Logical Block Address) and executing the `pwrite()` system call on the raw block device.

Read. For read operations, the requester node sends an RPC request to retrieve the file mapping information from the remote node, which queries the file mapping information from the file system using the Extent Handler and returns it to the requester node. Then, similar to the write process, the requester node uses the file mapping information and issues the `pread()` system call to directly read data from the disaggregated raw block device. It is noteworthy that unless there is significant memory pressure, the `ioctl(fiemap)` does not result in disk I/O since the file mapping is like to have been cached in the remote node's file system cache.

RDIO provides shared access to files on remote storage while still leveraging the existing Linux native file system. The remote data transfer process in RDIO is divided into metadata and data planes, ensuring that data is transmitted only once across the network fabric, unlike the traditional RPC-based access. Specifically, while data is transmitted through the storage network protocol, its metadata (e.g., file mapping information) is retrieved between nodes via RPC communication. Despite RPC communication on the metadata plane, since metadata is much smaller than the actual data, RDIO can reduce CPU usage and network traffic during remote data transfer.

Improving Metadata Plane for File Mapping

Every read or write request requires file mapping information, and each I/O operation must communicate with the remote node via RPC to access the file system's metadata. Although the size of the metadata is not large, there may be issues with increased I/O latency. To address this problem, RDIO implements *FileMap Cache*, a cache for file mapping information. The FileMap Cache eliminates one network hop in the metadata plane, allowing only the data plane to be performed through cached entries. As a result, when a cache hit occurs in Node A's FileMap Cache, the RPC request for file mapping is skipped, and the data can be directly read from the remote volume.

Entries in the FileMap Cache consist of the file path and file mapping information. As a metadata cache that does not store actual data, it has a small memory footprint, making it suitable for environments handling large datasets. Additionally, the FileMap Cache takes advantage of temporal locality for frequently re-accessed entries by applying the LRU cache replacement policy.

Cache consistency in distributed systems is crucial to ensure that all nodes have a consistent view of the data. On the other hand, there exist various modern applications that allow for weak cache consistency models. In distributed object stores such as MinIO, which consider the immutability of objects, complex cache consistency mechanisms used in generic shared disk file systems are not necessary. If objects

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on November 25,2025 at 02:38:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Figura 18: Han et al. (2025), *IEEE Micro*, página 3. **Fragmento subrayado:** «*The remote data transfer process in RDIO is divided into metadata and data planes... while data is transmitted through the storage network protocol, its metadata (e.g., file mapping information) is retrieved between nodes via RPC communication. Despite RPC communication on the metadata plane, since metadata is much smaller than the actual data...*». Citado y analizado en la Sección 5.3 (*Modelos de Comunicación*) [4].

the slave servers are combined into a single unit of data. In addition, even if a server fails and becomes unresponsive, the data stored on other servers can be used to recover the lost data [3]. However, due to the cost of distributing and storing data on each server, EC techniques have emerged as an alternative solution to increase the space efficiency of distributed storage systems. Unlike the replication technique, the EC technique uses a unique algorithm to encode the original data and create a parity block. The data block and the parity block are then distributed and stored on each server. Although the EC technique improves symmetrical space efficiency compared to the replication technique, there is an overhead when writing or reading data due to the involvement of many servers and disks [4]. Regarding the overhead, there have been studies on client overhead to solve disk Input/Output (I/O) problems as well as studies on solving the problem of data bottlenecks in terms of network traffic [5,6]. To address the issue of client overhead, this paper proposes a method to maximize the efficiency of the matrix by uploading the matrix generated during encoding and decoding through the EC algorithm to the cache memory, allowing for faster access.

The methodology of cache-based matrix technology applies the WSCRP algorithm for encoding and decoding [7]. The WSCRP algorithm operates based on the Weighting Replacement Policy (WRP) algorithm, which proposes a page replacement policy to be used in cache memory by introducing a new parameter called weights [8]. The WSCRP algorithm proposes a page replacement policy that extends the weights of the WRP algorithm by adding a new parameter called cost. The experiment applied a distributed file system, HDFS, which supports the EC algorithm. The EC algorithm selected for the experiment was Reed–Solomon (RS), and a (6, 3) volume was used, consisting of six data blocks and three parity blocks. The experimental evaluation targeted systems without cache memory, systems with Least Recently Used (LRU) and Least Frequently Used (LFU) basic cache memory algorithms, and systems with WRP and WSCRP algorithms, comparing a total of five writing times, reading times, and recovery times.

This paper begins with an introduction in Section 1 and examines the basic theory of distributed file systems in Section 2. Section 3 provides a summary of the improved methodologies in EC-based distributed file systems, along with related studies. Section 4 introduces the proposed cache-based matrix technology. In Section 5, the paper compares systems that applied the methodology to the distributed file system. Finally, Section 6 presents our conclusion.

2. Background

HDFS is a representative distributed file system, provided in an open-source format through the Apache Project, and many companies use it to build services. In a replication method of distributed file systems, such as HDFS, the write mode divides the original data into blocks, and the divided blocks are replicated and stored in each server. For instance, in replication techniques with 3x Factor elements, storing 300 GB (Gigabyte) of data requires 900 GB of storage space. Although data stability is increased in preparation for server failure, there is a disadvantage in that more physical capacity is required for symmetrical space efficiency. Figure 1 shows how data are written when nine nodes (servers) are configured through replication techniques.

When the storage command is given to the original data from the name node, the data are divided into blocks, and each block is replicated and written to the data node.

Distributed file systems that use EC instead of replication techniques are widely used to address symmetrical space efficiency aspects [9]. EC is a method of encoding and storing data using a unique algorithm. Representative algorithms exist in various forms, including XOR, RS, Liberation, and Weaver Code [10–13]. When data are written to the EC-based distributed file system, the original data are divided into data blocks and parity blocks through encoding. When data are read or recovered, data loss can be prevented in case of server failure through decoding. If the data are encoded using the RS (6, 3) volume, the original data are divided into six data blocks and encoded using the RS algorithm to

Figura 19: Shin & Kim (2023), *Symmetry*, página 2. **Fragmento subrayado:** *«However, due to the cost of distributing and storing data on each server, EC techniques have emerged as an alternative solution to increase the space efficiency of distributed storage systems. Unlike the replication technique, the EC technique uses a unique algorithm to encode the original data and create a parity block...»*. Citado y fundamentado en la Sección 5.4 (*Algoritmo Reed-Solomon*) [2].

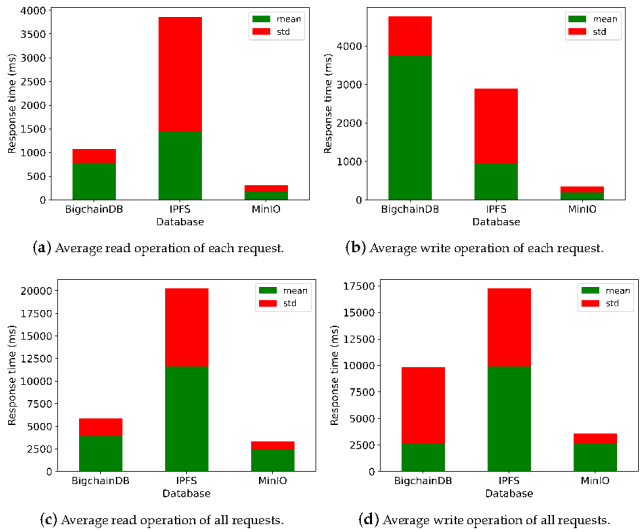
negligible. This proves that the storage is lightweight enough to be deployed on most edge devices, including the Raspberry Pis used for the evaluation. Figures 2 and 3 illustrate the statistics for the read and write operations, respectively. Figures 2a and 3a indicate the percentage of the RAM usage where, as depicted, MinIO consumes the least amount of RAM in each case. In addition, BigchainDB follows MinIO, only in the case of a single request, with the IPFS is ahead of BigchainDB in all users' requests.

In the rest of the figures where the disk latency (Figures 2b and 3b) and the disk IO time (Figures 2c and 3c) are presented, MinIO achieves the best performance followed by BigchainDB, while the IPFS yields the worst performance results. The disk metrics are increased by a larger degree, proving intense I/O activity.

Finally, Figure 4 demonstrates the transaction rate (TR) achieved by each storage system. The transaction rate can be defined as:

$$\mathrm{TR} = \frac{\sum \text{transactions}}{\text{total_time}} \quad (1)$$

As the results suggest, MinIO achieves the highest TR followed by BigchainDB, while the IPFS exhibits the worst results. For instance, the TR obtained by MinIO is 3.3 and 1.3 times larger compared to the IPFS and BigchainDB, respectively.



(a) Average read operation of each request.

(b) Average write operation of each request.

(c) Average read operation of all requests.

(d) Average write operation of all requests.

Figure 1. Performance of read/write operations of each database.

Figura 20: Makris et al. (2022), *Applied Sciences*, página 6. **Fragmento subrayado:** *«This proves that the storage is lightweight enough to be deployed on most edge devices... indicate the percentage of the RAM usage where, as depicted, MinIO consumes the least amount of RAM in each case... MinIO achieves the best performance followed by BigchainDB... transaction rate (TR) achieved by each storage system...»*. Citado y contrastado en la Sección 7.2 (*Análisis de Consumo*) y Sección 9 (*Fortalezas Demostradas*) [7].